

Inside X's For You Algorithm

A source-grounded breakdown of the May 2026 open-source release

Architecture, retrieval, ranking, candidate isolation, and the limits of what the public code can actually tell us.

AUTHOR Emil Veliyev

PRIMARY SOURCE xai-org/x-algorithm @ e414c17

SOURCE DATE May 15, 2026

REVISION October 2026

Scope note. This report analyzes the May 15, 2026 source snapshot. X published substantial additional code in August and September 2026, including production-facing Phoenix code, explicit score weights, and a broader visibility-filtering stack. Those later releases are used here only to clarify the limits of the May snapshot; they are not folded back into the historical architecture as if they were already present.

This is an independent technical analysis. It is not affiliated with or endorsed by X Corp. or xAI.

EXECUTIVE SUMMARY

What the May release actually showed

The May 2026 release of X's open-source recommendation code is most useful when read as a systems-engineering snapshot, not as a recipe for “beating the algorithm.” It shows how a For You request moves through an orchestration layer, draws candidates from in-network and out-of-network sources, enriches and filters those candidates, predicts several kinds of user response, and finally selects a ranked set of posts. [1][2]

Three parts deserve special attention. First, Thunder and Phoenix solve different retrieval problems: Thunder keeps recent posts from followed accounts close at hand, while Phoenix uses learned representations to search beyond the viewer's network. Second, Phoenix ranking is not a single engagement counter. The model predicts multiple actions per candidate, and a downstream scorer combines predicted values. Third, the ranker uses a candidate-isolation attention mask so that one candidate cannot change another candidate's representation simply by appearing beside it in the same inference batch. [2][3][4]

The May release also made the system easier to reproduce. It added an end-to-end retrieval-to-ranking script, a frozen mini-model and corpus distributed through Git LFS, a Grox content-understanding service, new candidate sources and hydrators, and an ads-blending path. Those additions made the repository far more useful for study, even though the public snapshot still left important production details out. [1][2]

The central takeaway is less dramatic than most social-media advice suggests: the public code describes a per-viewer recommendation pipeline built around predicted behavior, retrieval quality, eligibility checks, and diversification. It does not support simple statements such as “one reply is worth N likes” or “this one trick guarantees reach.”

Reading guide

Section	Question answered
1. Scope and method	What source snapshot is being analyzed, and what counts as evidence?
2. Request path	How does a feed request move through Home Mixer?
3. Retrieval	What do Thunder and Phoenix each contribute?
4. Phoenix ranking	How does candidate isolation work, and why does it matter?
5. Scoring	What do action weights mean - and what do they not mean?
6. Grox and blending	Where do content understanding and ads fit?
7. Reproducibility	What can a researcher actually run from the May release?
8. Limits	What remains unknown or changed in later releases?

1. Scope and method

The primary source for this report is commit `e414c17` of `xai-org/x-algorithm`, pushed on May 15, 2026. That commit added the May release notes, the combined Phoenix inference pipeline, the Grox service, ads-related code, additional hydrators and sources, and substantial updates to the candidate-pipeline framework. A second commit later the same day, `0bfc279`, replaced the Git LFS artifact pointer for the Phoenix bundle; it changed the packaged artifact, not the architecture described here. [1][6]

The analysis follows a simple rule: a claim is presented as fact only when the May repository states it directly or the relevant code makes it observable. Interpretations are labeled as such. Later X documentation is used only where it helps distinguish a May-era limitation from a fact that became visible later. This matters because the current repository is materially more complete than the May snapshot. [5]

1. **Direct source evidence.** Repository documentation, source files, function behavior, and configuration surfaces at the pinned commit.
2. **Engineering interpretation.** Reasoned conclusions about why a design choice is useful - for example, why candidate isolation improves score independence across batches.
3. **Not inferred.** Traffic volume, global request counts, hidden production thresholds, or confidential ranking behavior that the open repository does not establish.

One concrete documentation inconsistency is worth noting: the root May README describes a 256-dimensional, 2-layer mini Phoenix artifact, while `phoenix/README.md` describes a 128-dimensional, 4-layer mini configuration. Because the artifact configuration is loaded at runtime, this report treats those numbers as demo-package details rather than production specifications. [2][3]

2. The request path: orchestration before ranking

Home Mixer is the coordinating layer. It does not simply hand a pile of posts to a neural network. A request is hydrated with viewer context, candidate sources run, candidate metadata is filled in, filters remove ineligible items, scorers assign values, a selector chooses the highest-ranked candidates, and post-selection checks run before the final response is returned. The candidate-pipeline abstraction makes those stages explicit. [2]

Stage	Role in the May pipeline
Query hydration	Build viewer context such as recent actions, following information, topic context, served history, and other request features.
Candidate sources	Fetch possible posts from sources such as Thunder and Phoenix, alongside newer May sources.
Candidate hydration	Attach post, author, language, media, engagement, and other metadata needed downstream.
Pre-scoring filters	Remove candidates that should not proceed to ranking.
Scorers	Generate model predictions and combine them with additional scoring logic.
Selector	Order candidates and keep the requested result set.
Post-selection processing	Run final hydration, visibility/dedup checks, and asynchronous side effects.

The framework also runs independent work in parallel where possible. In practice, that means the architecture is as much about controlling latency and failure boundaries as it is about model quality. The model sits inside a larger serving system; it is not the system by itself. [1][2]

A simplified May 2026 request path

01	For You request
02	Home Mixer: hydrate viewer context
03	Candidate retrieval: Thunder + Phoenix + other enabled sources
04	Hydrate candidates and apply eligibility filters
05	Phoenix predictions → weighted scoring → diversity / OON adjustments
06	Select top candidates and run post-selection checks
07	Ranked feed response

3. Retrieval: Thunder and Phoenix solve different problems

Thunder: recent in-network content

Thunder is described in the May repository as an in-memory post store fed by post create/delete events. It maintains recent material on a per-user basis and supplies in-network candidates - posts from accounts the viewer follows. The project documentation describes the lookup path as sub-millisecond and emphasizes avoiding a database round trip for this stage. [2]

That design fits the problem. In-network retrieval does not need to search the whole corpus for conceptual similarity; it needs a fast answer to a much narrower question: what recent posts are available from this viewer's network?

Phoenix retrieval: searching beyond the network

Phoenix retrieval addresses the opposite case. A two-tower model builds a representation for the viewer and representations for candidate posts, then uses similarity search to pull a manageable set of out-of-network candidates from a much larger corpus. In the released demo, retrieval uses dot-product similarity over precomputed candidate representations. [3][7]

The two-stage pattern is standard for large recommendation systems for a reason: a relatively cheap retrieval stage reduces millions of possibilities to a smaller set, and a more expressive ranker spends compute only where it can matter. The open-source Phoenix demo makes that separation unusually easy to inspect. [3]

4. Phoenix ranking and candidate isolation

The ranking model consumes three broad groups of information: a user representation, a sequence of past interactions, and the candidates to be scored. It produces action logits for each candidate. The distinctive implementation detail is the attention mask created by `make_recsys_attn_mask` in `phoenix/grok.py`. [4]

The mask lets each candidate attend to the viewer context and to itself, but not to other candidates. In plain terms, candidate A can use what the model knows about the viewer, but it cannot change its score just because candidate B happens to be placed beside it in the same batch. [4]

Candidate-isolation mask, simplified

Query \ Key	Viewer + history	Candidate A	Candidate B	Candidate C
Viewer + history	context	-	-	-
Candidate A	yes	self	no	no
Candidate B	yes	no	self	no
Candidate C	yes	no	no	self

Why is this useful? The strongest reason is score independence. If a candidate's representation depended on the other candidates in the batch, the same post could receive a different model score simply because its batch companions changed. Candidate isolation removes that interaction. It also removes the candidate-to-candidate quadratic attention block, though it would be too strong to say that the entire transformer becomes “linear” in every dimension; the viewer/history context still has its own attention cost. [3][4]

This is a good example of where careful wording matters. The code clearly removes cross-candidate attention. That is stronger and more defensible than turning the implementation detail into a sweeping claim that all inference complexity has become linear.

5. Multi-action scoring: predicted behavior, not engagement arithmetic

Phoenix is built to predict several possible responses to a candidate rather than one generic relevance number. The May mini configuration exposes 19 action types, and the documentation lists outcomes such as favorite, reply, repost, quote, click, profile click, video view, photo expand, share, dwell, follow, not interested, block, mute, and report. [2][3]

A downstream weighted scorer combines model outputs into a ranking score. The important word is *predicted*. The weights apply to the model's estimate of what this viewer is likely to do; they are not exchange rates between raw engagement counts. X's August 2026 documentation later made this point explicit because readers repeatedly interpreted weights as statements such as “one report cancels 468 likes.” That is not how the score works. [5]

The May end-to-end demo is another place where overreading is easy. `run_pipeline.py` includes a small illustrative formula that combines favorite, reply, repost, and dwell probabilities with simple coefficients. Those coefficients are part of the demonstration harness. They should not be presented as the production For You weights. [7]

What you can safely conclude

- The ranker is multi-task: it estimates multiple user actions for each candidate.
- The final ordering can combine positive and negative predicted outcomes rather than optimizing one engagement event in isolation.
- The score is personalized because the probabilities depend on the viewer context and recent history.
- The May public snapshot is not a reliable basis for converting one observed engagement into a universal number of another engagement.

What you should not conclude

- A reply has a fixed universal value relative to a like.
- A visible coefficient in a demo script is necessarily a production coefficient.
- A post carries one global reach score that is independent of the viewer.

6. Filtering, content understanding, and blending

Eligibility and post-selection checks

The May pipeline documents filters for duplicates, age, self-posts, repost deduplication, subscription eligibility, previously seen or served posts, muted keywords, and author social-graph constraints. A post-selection visibility filter and conversation deduplication step appear after ranking. This is a reminder that ranking is only one gate in the serving path. A high model score does not override eligibility. [2]

Grox: asynchronous content understanding

The May release introduced Grox as a separate content-understanding service with classifiers, embedders, and task execution for workloads including spam detection, post-category classification, and PTOS policy enforcement. That architecture keeps expensive or asynchronous understanding work separate from the latency-critical request path while still allowing derived labels and embeddings to be consumed downstream. [1][2]

It is tempting to read every classifier or label as a direct ranking penalty. The public architecture does not justify that shortcut. Some signals are used for eligibility, safety, brand-safety, categorization, or candidate hydration rather than as a simple additive rank term. Later releases made that separation between ranking and visibility filtering much clearer. [5]

Ads and other non-post items

The May code also added an ads-blending module and new non-post candidate paths. These components matter because the final feed is not produced by one monolithic ranking list. Home Mixer coordinates post ranking and then blends or positions other items under separate constraints, including brand-safety considerations. [1]

7. Reproducibility: what the May release lets you run

The most practically valuable part of the May drop is `phoenix/run_pipeline.py`. It loads exported retrieval and ranking checkpoints, a precomputed corpus, and a user action sequence, then runs retrieval followed by ranking. The default demo retrieves the top 200 candidates from the packaged corpus before scoring them in chunks supported by the ranker. [7]

The supplied sports corpus is a demonstration environment rather than a mirror of the full X corpus. `phoenix/README.md` describes roughly 537,000 sports-related post IDs from a six-hour window and a frozen mini-model. Production Phoenix is described as larger and continuously trained. The public artifact is therefore best viewed as a reproducible teaching and research object, not a complete local clone of X's live feed. [3]

A sensible experiment

1. **Hold the corpus fixed.** Use the same demo corpus so retrieval candidates remain comparable.
2. **Change one part of the user history.** For example, alter one interaction or action value instead of replacing the whole sequence.
3. **Compare retrieval and ranking separately.** A post can move because it was retrieved differently, because its predicted actions changed, or both.

4. **Repeat across several histories.** One synthetic user sequence is not evidence for a general rule about all users.

The demo is strongest as a controlled laboratory. It is weakest when treated as a calculator that reveals production reach for a real account.

8. What the May snapshot does not tell us

Open-source code is unusually informative, but it has boundaries. The May repository itself says the Phoenix implementation is representative of the internal model except for specific scaling optimizations. It also exposes a frozen mini checkpoint rather than continuous production training. Those qualifications should be carried into any conclusion drawn from the code. [3]

Open source supports	Open source does not establish
The staged request path and candidate-pipeline structure.	The exact traffic volume or number of ranking decisions made globally.
Thunder/Phoenix roles and the two-stage retrieval/ranking design.	All closed services, datasets, labels, or experiments that may affect production.
Candidate-isolation attention behavior in the released Phoenix model.	That every production scaling optimization is identical to the demo.
Multi-action prediction and weighted combination as a scoring pattern.	A permanent universal exchange rate between likes, replies, reports, or other actions.
The existence of Grox, ads blending, hydrators, filters, and released demo artifacts.	That every visible field is a direct ranking signal or that every absent field is irrelevant.

Later releases reinforce this caution. On August 13, X published additional production-facing Phoenix code, explicit configuration parameters including weights, and a much broader visibility-filtering and labeling stack. On August 14, the project documentation explicitly corrected common misunderstandings about action weights. On September 18, the transparency tooling gained additional legal-compliance reporting. The May snapshot remains valuable, but it should be cited as a historical snapshot rather than described as the complete current algorithm. [5]

Conclusion

The May 2026 X algorithm release is interesting because it makes a modern recommendation stack legible end to end. The architecture separates fast in-network retrieval from learned out-of-network retrieval, uses a transformer to predict several kinds of viewer response, isolates candidates during ranking, and embeds the model inside a larger pipeline of hydration, filtering, selection, and side effects. [2][3][4]

The most durable lesson is methodological. Recommendation systems this large are not well described by a list of folk rules. The code points instead to a set of interacting systems: what is eligible, what is retrieved, what the model predicts for a particular viewer, how those predictions are combined, how repeated authors are handled, and what final checks apply before serving. Any explanation that collapses all of that into a single magic weight is easier to remember, but less faithful to the source.

Read this release as an engineering snapshot. Use the runnable model to test ideas. Keep observations separate from inference. And when later source releases contradict an earlier interpretation, update the interpretation rather than defending the story.

References and source notes

- [1] xai-org/x-algorithm, commit e414c171ed68266341193330bc4864bf3f3534e3, “Open-source X Recommendation Algorithm,” May 15, 2026. <https://github.com/xai-org/x-algorithm/commit/e414c171ed68266341193330bc4864bf3f3534e3>
- [2] xai-org/x-algorithm, README.md at e414c17, May 15, 2026. <https://github.com/xai-org/x-algorithm/blob/e414c171ed68266341193330bc4864bf3f3534e3/README.md>
- [3] xai-org/x-algorithm, phoenix/README.md at e414c17. <https://github.com/xai-org/x-algorithm/blob/e414c171ed68266341193330bc4864bf3f3534e3/phoenix/README.md>
- [4] xai-org/x-algorithm, phoenix/grok.py at e414c17, including make_recsys_attn_mask. <https://github.com/xai-org/x-algorithm/blob/e414c171ed68266341193330bc4864bf3f3534e3/phoenix/grok.py>
- [5] xai-org/x-algorithm, current README.md, notable updates dated August 13-14 and September 18, 2026. Accessed October 1, 2026. <https://github.com/xai-org/x-algorithm>
- [6] xai-org/x-algorithm, commit 0bfc2795d308f90032544322747caacd535f75ae, Git LFS artifact pointer update, May 15, 2026. <https://github.com/xai-org/x-algorithm/commit/0bfc2795d308f90032544322747caacd535f75ae>
- [7] xai-org/x-algorithm, phoenix/run_pipeline.py at e414c17. https://github.com/xai-org/x-algorithm/blob/e414c171ed68266341193330bc4864bf3f3534e3/phoenix/run_pipeline.py

Revision note

This revised edition replaces a denser earlier presentation. The technical scope remains centered on the May 2026 open-source release, but claims have been narrowed where the source does not support production-scale certainty, and later official documentation is cited where it materially changes how the May snapshot should be interpreted.